

KAREL – LEKTION 1

Was ist Programmieren?



Nun wissen wir alles darüber was wir brauchen um SparkPay zu entwickeln. Doch wie Entwickelt man eine App? Um eine App zu erstellen, müssen wir lernen zu programmieren. “Ich habe einen Freund mitgebracht” sagt Sparky mega begeistert. Karel ist kein gewöhnlicher Roboter... Er arbeitet als Archäologe in einem uralten Tempel voller Geheimnisse, Fallen und verlorener Schätze.

Leider gibt es ein Problem:

- 👉 Der Tempel ist zu gefährlich für Menschen.
- 👉 Deshalb wurde Karel geschickt.

Noch ein Problem:

- 👉 Karels Chef hat absolut keine Ahnung von Programmieren 🤖.

🧠 Was ist Programmieren?

Programmieren bedeutet: Einem Computer (oder Roboter) ganz genaue Anweisungen zu geben.

Wichtig:

- Computer verstehen keine Vermutungen
- Computer verstehen nur exakte Befehle

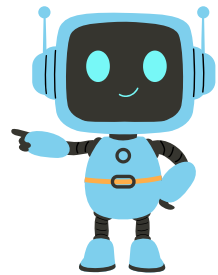
Aufgabe 1: Schreibe eine Anleitung für einen Roboter: „Wie macht man ein Sandwich?“ Schreibe jeden einzelnen Schritt auf Sei so genau wie möglich! (z.B. Toaster muss erst angeschaltet werden, dafür den Stecker nicht vergessen 😊)

Mission 1 - Let there be Karel

Ein leises Summen erfüllt die Luft. Nach tausenden Jahren wird der Tempel wieder betreten. Der Roboter Karel soll aktiviert werden. Hilf ihm dabei:

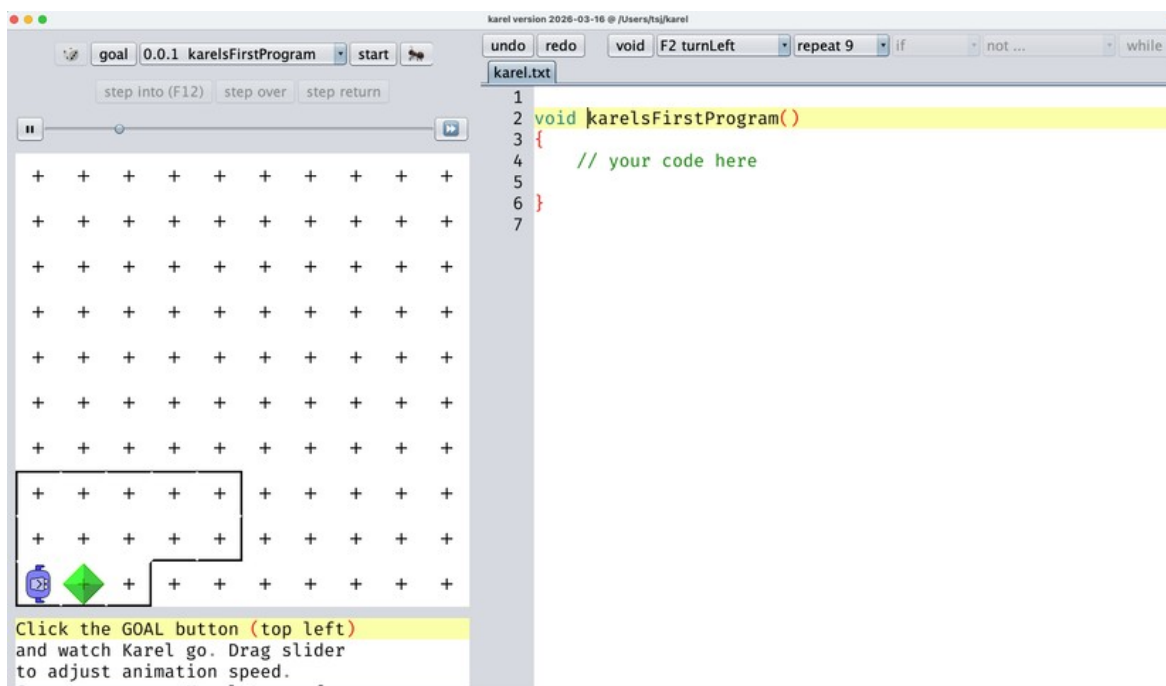
Installation

1. Lade zuerst Karel herunter: Klicke auf „karel.jar“
2. Öffne deinen Downloads-Ordner (über den Explorer bei Windows oder Finder bei macOS).
3. Doppelklicke auf die Datei „karel.jar“
4. Karel startet → Perfekt! Du bist fertig. 



Achtung!:  **Nur** wenn nichts passiert oder eine Fehlermeldung erscheint, Lies weiter ↓

1. Klicke auf „Download Java“, und lade die passende Java Version herunter.
2. Warte, bis der Download abgeschlossen ist.
3. Öffne erneut deinen Downloads-Ordner
4. Doppelklicke auf „jre...“, um die Installation zu starten.
5. Klicke dich durch die Installation: Weiter → Weiter → Installieren → Fertigstellen
6. Öffne nun nochmal die Datei „karel.jar“



Solltest du dieses Fenster sehen, wurde alles richtig installiert.

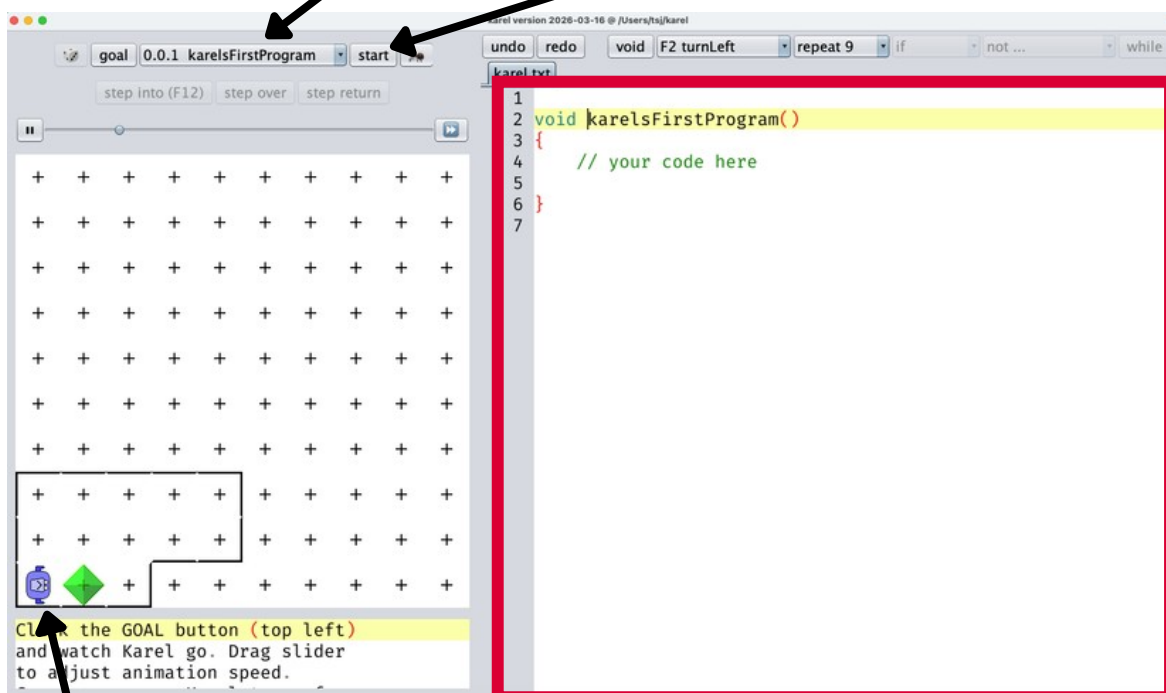
Funktionen der Umgebung

Herzlichen Glückwunsch, du hast nun Karel The Robot installiert. Lass uns nun die Umgebung (App/Anwendung) zusammen erkunden:

Goal: zeigt an, was Karel mit dem Code schaffen soll

hier kannst du das Level auswählen

hiermit führst du den Code aus. Mit dem Käfer siehst du sofort ob alles funktioniert



Und hier befindet sich Karel selbst auf seinem Spielfeld :)



Hier schreibst du deine Anweisungen an Karel

Nun ist es an der Zeit die ersten Anweisungen an Karel zu schreiben. Dazu mehr im nächsten Kapitel.

Mission 2 - Befehle kennenlernen

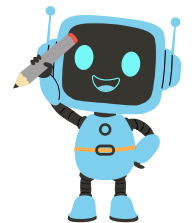
Nun kannst du Karel bestimmte Befehle geben. Dabei stehen dir erstmal 5 Befehle zur Verfügung:

```
moveForward();      // Karel geht einen Schritt nach Vorn
turnLeft();         // Karel dreht sich nach links
turnAround();       // Karel dreht sich um 180 Grad
turnRight();        // Karel dreht sich nach rechts
pickBeeper();       // Karel hebt einen Beeper auf
dropBeeper();       // Karel legt einen Beeper ab

// Dies erzeugt einen Kommentar
```

Aufgabe: Benutze diese Befehle um die Aufgaben:

- 0.0.1 karelsFirstProgram
- 1.1.1 obtainArtifact
- 1.1.2 defuseOneBomb
- 1.1.3 defuseTwoBombs
- 1.1.4 practiceHomeRun



zu bearbeiten. Bei Fragen oder wenn du nicht weiterkommen solltest, melde dich gerne bei mir. Viel Erfolg :)

Tipp: jede Aufgabe startet mit dem Befehl Void und dem Namen des Levels (also das, was hinter den Zahlen steht). So schreibst du für das Level 1.1.1 obtainArtifact den folgenden Code:

```
void obtainArtifact() {

    // Deine Anweisungen schreibst du hier :- )

}
```

Mission 3 - Schleifen



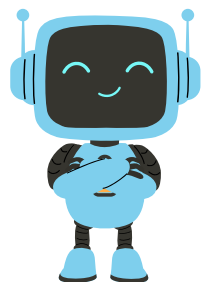
Karel bewegt sich durch den Tempel und muss lange Gänge entlanglaufen. Bisher wurde jeder Schritt einzeln programmiert (hoffentlich 😊). Das funktioniert, ist aber unübersichtlich und aufwendig, wenn man viele Schritte oder Wiederholungen programmieren muss. So müssten wir bei 10 Schritten vorwärts schreiben (Bild 1). Wie wäre es, wenn wir dem Roboter sagen könnten "Wiederhole 10 mal `moveForward()` ; ". Dies geht mit einer Schleife (Bild 2).

```
moveForward();  
moveForward();  
moveForward();  
moveForward();  
moveForward();  
moveForward();  
moveForward();  
moveForward();  
moveForward();  
moveForward();
```

Bild 1

```
repeat (10)  
{  
    moveForward();  
}
```

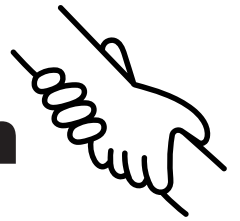
Bild 2



Eine Schleife wird im Programmieren immer dann verwendet, wenn etwas oft wiederholt werden soll.

Aufgabe 2: Überarbeite deine bisher gelösten Aufgaben und versuche wiederholungen in Schleifen umzuwandeln.

Aufgabe 3: Warum ist es praktisch, Schleifen zu verwenden?



Mission 4 - Hilfsfunktionen

Nachdem du Karel durch den Tempel bewegt hast, ist dir vielleicht aufgefallen:

- Einige Bewegungen oder Schleifen wiederholen sich genau gleich
- Du schreibst denselben Code mehrfach hintereinander

Dieses Problem lässt sich mit Hilfsfunktionen lösen. Eine Hilfsfunktion (oder „Methode“) fasst einen wiederkehrenden Codeblock zusammen, den man mehrfach aufrufen kann.

Vorteile:

- Code ist kürzer und übersichtlicher
- Änderungen müssen nur einmal gemacht werden
- Fehler werden reduziert
- Programme werden strukturierter



So würde man statt den Code mit Wiederholungen (Bild 3) in eine Hilfsfunktion (Bild 4) auslagern:

```
repeat(3) {
    moveForward();
}
turnLeft();

repeat(3) {
    moveForward();
}
turnLeft();
```

Bild 3

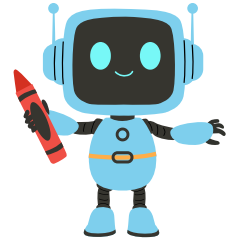
```
void gehe3SchritteLinks() {
    repeat(3) {
        moveForward();
    }
    turnLeft();
}
```

```
gehe3SchritteLinks();
// Karel führt die
// Hilfsfunktion aus
gehe3SchritteLinks();
// Karel führt die
//Hilfsfunktion erneut aus
```

Bild 4

Die Hilfsfunktion wird immer nach derselben Schablone gebildet:

```
void funktionsname() {  
  
    // Hier stehen die Anweisungen  
  
}
```

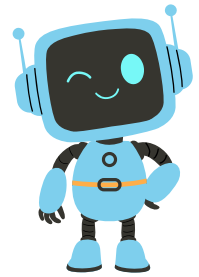


Aufgabe 4: Warum ist es praktisch, Hilfsfunktionen zu verwenden?

Aufgabe 5: Gehe nocheinmal die Aufgaben 0.* und 1.1* durch und überarbeite sie so, dass doppelte Schleifen oder Anweisungen nun in eine Hilfsfunktion verpackt werden. Achte darauf der Funktion einen Namen zu geben, der direkt erklärt, was diese Funktion tut.

Tipp: Oft ist es sinnvoll, Kommentare zu bestimmten Zeilen des Codes zu schreiben. Stell dir vor, du schaust den Code in 5 Jahren nochmal an. Verstehst du direkt, was du dir damals gedacht hattest und wie der Code funktioniert? Vermutlich nicht! Daher wäre es großartig Kommentare an wichtigen Stellen zu verfassen. In Karel machst du dies mit:

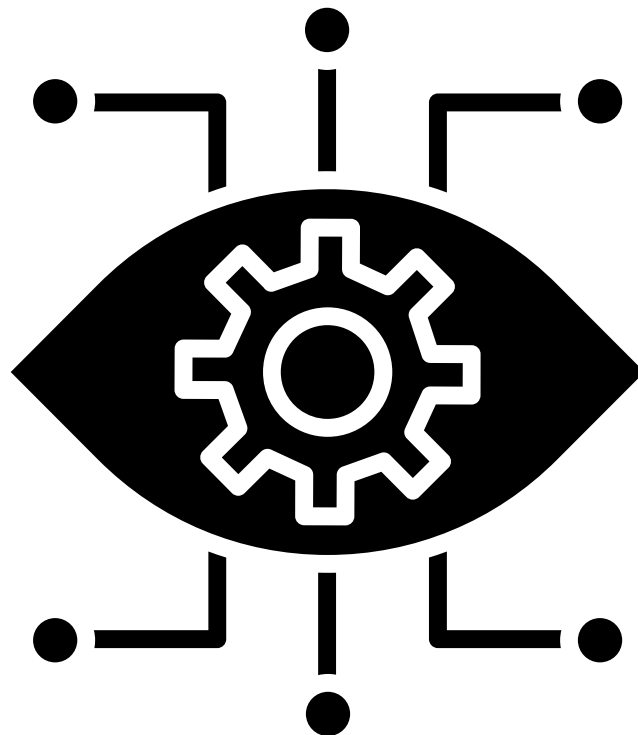
```
// Hier steht dein Kommentar (// leitet einen Kommentar ein)
```



Wie geht es weiter?

Nun bist du der Profi was Kommentare, Funktionen, Hilfsfunktionen und Schleifen in Karel angeht. Du kannst dich nun erstmal an den weiteren Aufgaben versuchen. Im Laufe der Zeit kommen neue Befehle und Herausforderungen hinzu. Versuch diese Aufgaben zu meistern. Viel Spaß bei den Aufgaben und melde dich gerne bei Fragen.

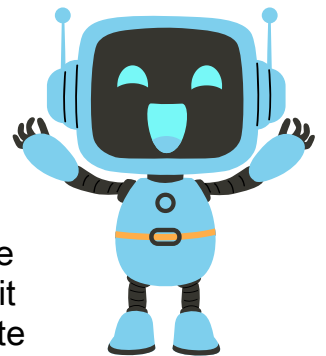
Wenn du bei Aufgabe 1.3.4 angekommen bist, kannst du zu Java Entwicklung wechseln (nächste Seite).



JAVA – LEKTION 1

Einstieg in Java

Bisher hast du mit Karel gearbeitet und dabei die wichtigsten Grundlagen des Programmierens kennengelernt. Du hast gelernt, wie man Probleme in einzelne Schritte zerlegt, Befehle wiederholt und mit Hilfsfunktionen strukturiert arbeitet. Diese Denkweise ist der wichtigste Teil des Programmierens – und genau darauf bauen echte Programmiersprachen auf. Jetzt gehst du den nächsten Schritt:



👉 Du programmierst nicht mehr nur einen Lernroboter, sondern arbeitest mit einer echten Programmiersprache 😎.

Java ist eine der weltweit am häufigsten verwendeten Programmiersprachen. Sie wird in vielen Bereichen eingesetzt, zum Beispiel für Apps, große Softwaresysteme und Anwendungen in Unternehmen.

Viele Konzepte, die du bereits mit Karel kennengelernt hast, findest du auch in Java wieder:

- Wiederholungen → Schleifen
- Hilfsfunktionen → Methoden
- Befehle → Anweisungen im Code

Der Unterschied ist: In Java hast du deutlich mehr Möglichkeiten – aber auch mehr Verantwortung, deinen Code sauber und korrekt zu schreiben. Mit anderen Worten: Du wechselst jetzt von einer Lernumgebung zu einem Werkzeug, das auch im Berufsleben eingesetzt wird.

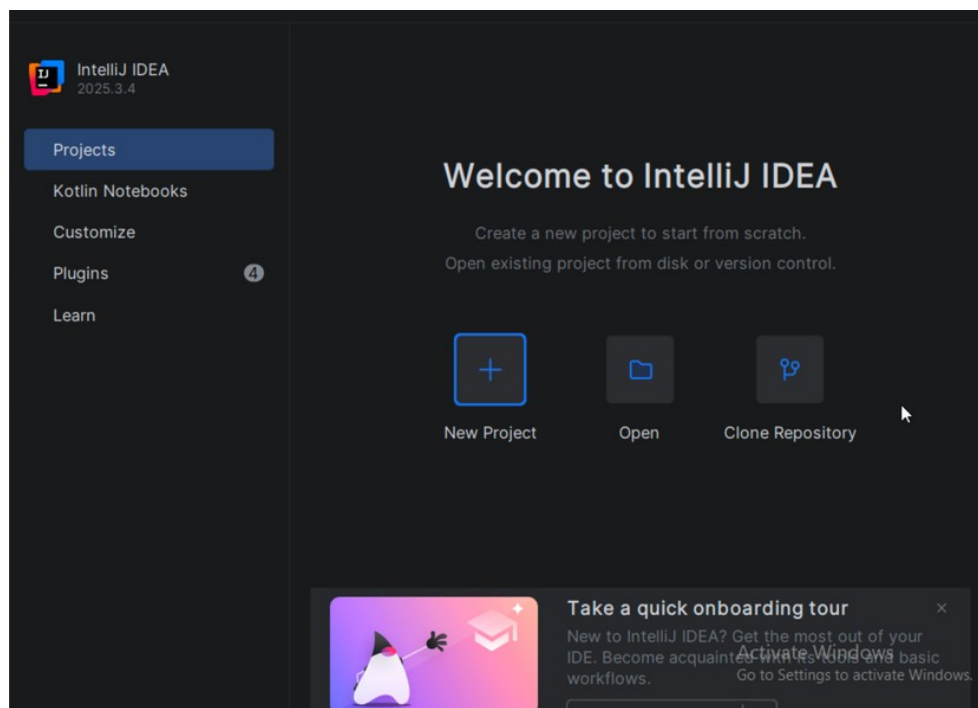
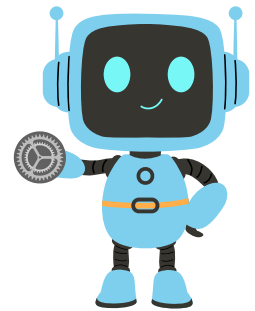
Ziel ist es am Ende die SparkPay – Banking Simulations-App zu bauen . Hilf Sparky dabei!

Java Einrichtung

Bevor Javinator seine ersten Aufgaben lösen kann, muss seine Entwicklungsumgebung eingerichtet werden – denn ohne diese kann kein Code ausgeführt werden.

Installation

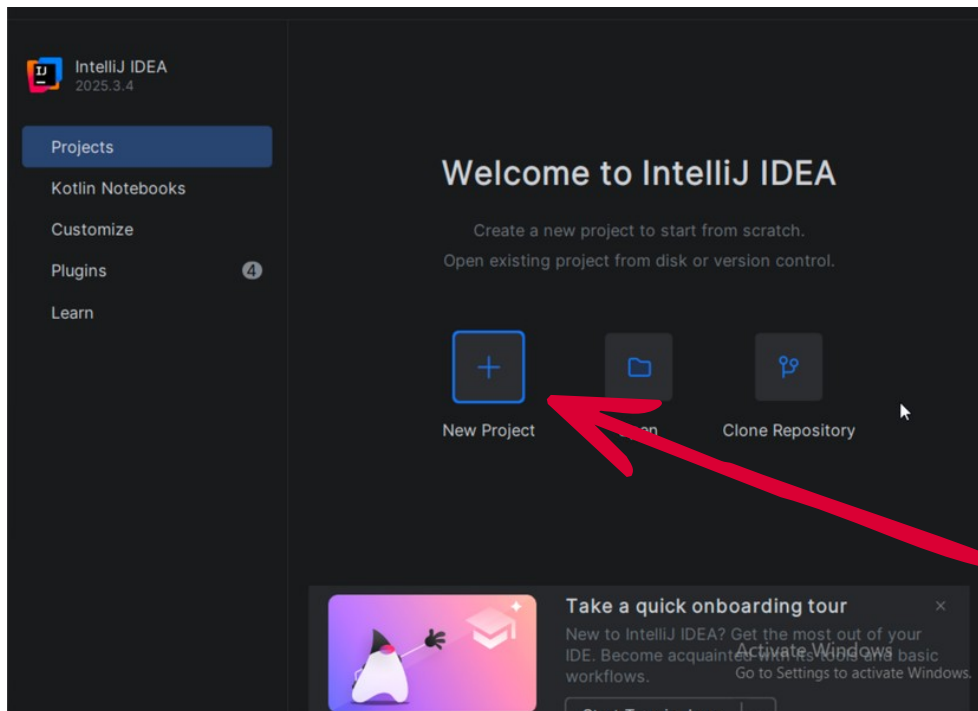
1. Lade zuerst IntelliJ IDEA herunter
2. Warte, bis der Download abgeschlossen ist.
3. Öffne deinen Downloads-Ordner (über den Explorer bei Windows oder Finder bei macOS).
4. Doppelklicke auf die heruntergeladene Datei, um die Installation zu starten.
5. Klicke dich durch die Installation: Next → (Create “Desktop Shortcut” u. “.java” aktivieren) , Next → Next → (JetBrains) Install → finish
6. Nun das Programm auf dem Desktop starten, dem User Agreement zustimmen und Data Sharing mit “Don’t send” beantworten.



Solltest du in etwa so ein Fenster sehen, wurde alles richtig installiert.

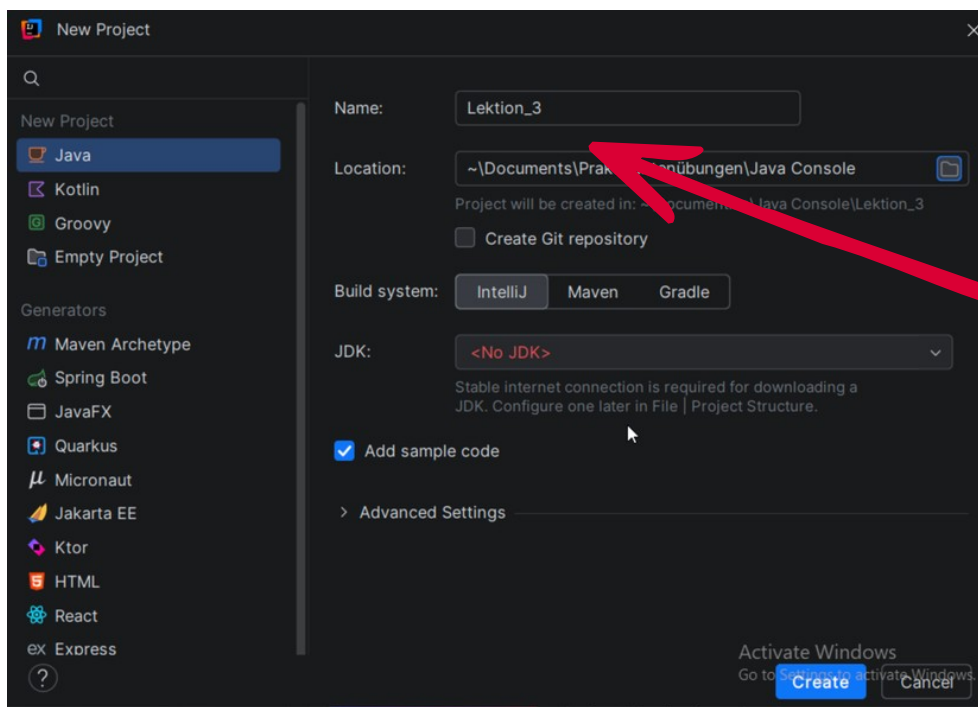
Funktionen der Umgebung

Herzlichen Glückwunsch, du hast nun die Umgebungsentwicklung mit Java heruntergeladen. Lass uns nun die Umgebung (App/Anwendung) zusammen erkunden und einrichten:



Über "New Project" kannst du ein neues Projekt anlegen.

Über "Open" bereits existierende Projekte öffnen und bearbeiten.

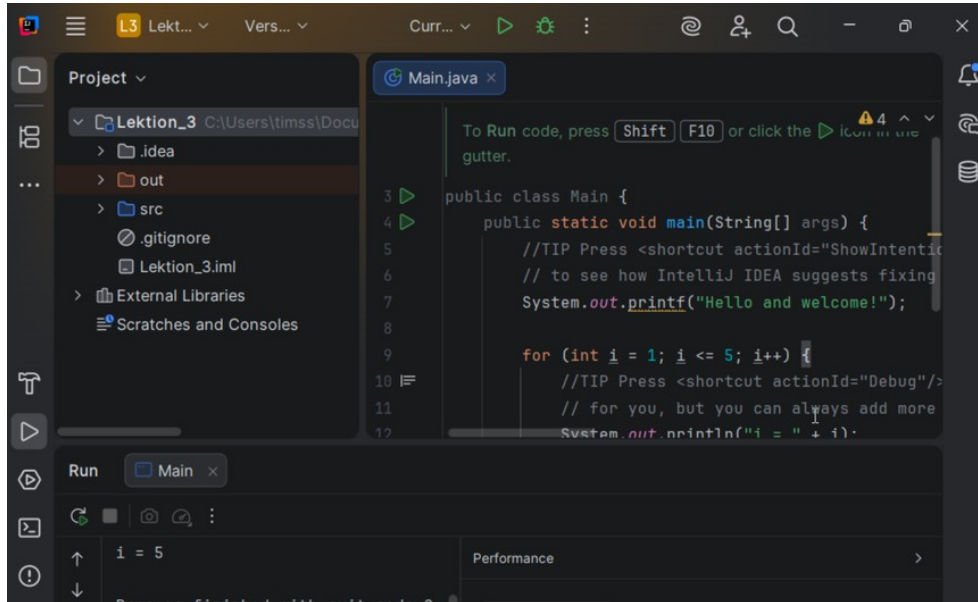


Namen eintragen, Speicherort auswählen (meistens einfach das vorgegebene wählen)

Bei JDK: BellSoft Liberica JDK wählen

Funktionen der Umgebung

Nun sollte ein Fenster mit Beispiel-Code auftauchen:

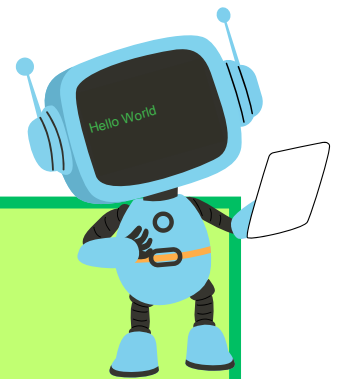


Mann sollte den Code ab //TIP Press bis hin zu der ersten } Klammer entfernen. Der verbliebene Code sieht dann folgendermaßen aus:

```
public class Main {  
    public static void main(String[] args) {  
  
        // Hier kommt dann dein Code hin  
  
    }  
}
```

Nun kannst du mit dem Coden beginnen :)

Konsolenausgabe



In der ersten Woche beginnt dein Praktikum bei der StarFinanz. Du lernst den Java-Roboter Javinator kennen, der dich bei deinen Aufgaben unterstützt. Gemeinsam sollt ihr ein System entwickeln, das Daten verarbeitet und anzeigt – ähnlich wie es auch in echten Anwendungen im Banking-Bereich benötigt wird.

Bevor komplexe Funktionen umgesetzt werden können, steht zunächst ein wichtiger erster Schritt an: Daten sichtbar machen. „Wir sollten mit etwas Einfachem beginnen“, erklärt Javinator. „Es ist sinnvoll, Informationen zunächst in der Konsole auszugeben.“

Du setzt dich an den Rechner und programmierst gemeinsam mit Javinator ein erstes kleines Programm. Ziel ist es, einfache Daten sichtbar zu machen und zu überprüfen, ob alles korrekt funktioniert. Kurz darauf erscheint auf dem Bildschirm: „Kontostand: 3.500 €“.

„Das ist ein guter Anfang“, sagt Javinator. „Jetzt können wir sicherstellen, dass unsere Ausgaben funktionieren. Darauf bauen wir alles Weitere auf.“

Ein einfacher Schritt – aber ein wichtiger Anfang. Denn bevor Programme Daten verarbeiten oder speichern können, müssen sie zuverlässig Ausgaben erzeugen.

Aufgabe 6: Warum ist es wichtig, dass ein Programm die Ausgaben von Daten zuverlässig in der Konsole anzeigt? Welche Folgen könnten entstehen, wenn das Banksystem der StarFinanz falsche Angaben macht (z.B. falschen Kontostand)?

Konsolenausgabe in Java

In der Programmierung ist es wichtig, dass Programme Rückmeldungen geben und Informationen anzeigen können. Eine einfache Möglichkeit dafür ist die Konsolenausgabe.

Dabei werden Daten im sogenannten Terminal oder in der Konsole angezeigt. In Java geschieht das mit:

```
System.out.println("Kontostand: 3.500 €"); // Gibt den Text in der Konsole aus
```

Dieser Befehl sorgt dafür, dass Text oder Werte auf dem Bildschirm erscheinen.

🧠 Wie funktioniert das?

- System: Eine Klasse, die in jedem Java-Programm verfügbar ist
- out: Die Standardausgabe (also die Konsole)
- println(): (Steht für Print **L**ine) gibt den Inhalt aus und fügt danach einen Zeilenumbruch hinzu

Beispiel mit Daten

```
double kontostand = 3500.00; // Speichert den Kontostand
System.out.println("Kontostand: " + kontostand + " €"); // Gibt ihn aus
```

Ausgabe in der Konsole: **Kontostand: 3500.0 €**

🎯 Warum ist das wichtig?

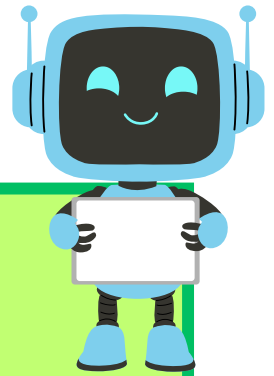
Die Konsolenausgabe hilft dir: Programme zu testen, Fehler schneller zu finden und zu verstehen, was dein Code gerade macht. Auch in echten Anwendungen (z. B. im Banking) ist es wichtig, dass Daten korrekt angezeigt werden.

Aufgabe 7: Was sind die Vorteile der Konsolenausgabe in der Programmierung?

Aufgabe 8: Programmiere gemeinsam mit Javinator ein erstes kleines Java-Programm. Ziel ist es, einfache Informationen in der Konsole auszugeben. Gib dazu die Texte „Willkommen bei der StarFinanz“ und „Kontostand: 1500 €“ jeweils in einer neuen Zeile aus. Verwende dafür die Methode `System.out.println()`. Optional kannst du den Kontostand in einer Variable speichern und anschließend ausgeben.

Tipp: Nenne das Java Projekt „Lektion 3“.

Primitive Datentypen und Variablen



Nachdem du gemeinsam mit Javinator erste Informationen in der Konsole ausgegeben hast, stellt sich ein neues Problem:

Bisher wurden alle Werte direkt in den Code geschrieben. Das funktioniert für einfache Beispiele, ist aber nicht flexibel.

Wenn sich der Kontostand ändert, müssen wir den Code jedes Mal anpassen“, erklärt Javinator. „Das ist in echten Anwendungen nicht praktikabel.“

Stattdessen brauchst du eine Möglichkeit, Daten zu speichern und wiederzuverwenden. „Dafür verwenden wir Variablen“, sagt Javinator. „So können wir Werte flexibel speichern und weiterverarbeiten.“

Variablen und Datentypen

Eine Variable ist ein Speicherplatz für Daten. Jede Variable hat einen Datentyp, der festlegt, welche Art von Wert gespeichert wird.

Wichtige Datentypen in Java sind: int (integer), double, boolean und String:

```
int menge = 3;           // Ganze Zahl (z. B. Anzahl)
double preis = 2.50;     // Dezimalzahl (z. B. Geldbetrag)
boolean aktiv = true;    // Wahrheitswert (true oder false)
String produkt = "Kaffee Latte"; // Text
```

Deklaration und Initialisierung

Variablen werden in Java meist in einer Zeile erstellt:

```
int menge = 3; // Variable wird erstellt und bekommt einen Wert
```

- Deklaration → Variable wird angelegt
- Initialisierung → Variable bekommt einen Wert

Warum sind Variablen wichtig?

Mit Variablen kannst du Daten speichern und wiederverwenden, Programme flexibler machen und Werte einfach ändern, ohne den ganzen Code anzupassen

👉 Genau das wird auch in echten Anwendungen (z. B. im Banking) benötigt.

Aufgabe 9: Überlege, warum es sinnvoll ist, Variablen zu verwenden.

⇒ Warum ist es praktischer, Daten wie Kontostand oder Beträge in Variablen zu speichern, anstatt sie direkt in den Code zu schreiben?

⇒ Welche Probleme könnten entstehen, wenn man keine Variablen verwendet?

Aufgabe 10:

a) Programmiere gemeinsam mit Javinator ein kleines Java-Programm für die Küche der StarFinanz. Speichere folgende Daten in Variablen und gib sie in der Konsole aus:

- Produkt: Kaffee Latte
- Menge: 3
- Preis: 2.50 €

b) (Erweiterung)

Berechne den Gesamtpreis und gib das Ergebnis in der Konsole aus:

👉 Gesamtpreis = Preis × Menge

Aufgabe 11: Warum ist der Datentyp double für Preise sinnvoll?

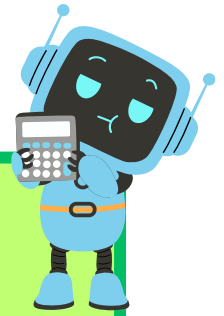
Aufgabe 12: Warum sollte ein Produktname als String gespeichert werden?

📌 Zusammenfassung

Variablen und Datentypen sind die Grundlage jeder Programmierung. Sie ermöglichen es, Daten strukturiert zu speichern und flexibel zu verarbeiten.

👉 Ohne Variablen wären Programme unflexibel und schwer erweiterbar.

Arithmetische Operationen



Nachdem du gemeinsam mit Sparky Daten in Variablen speichern kannst, entsteht das nächste Problem:

„Ein Kontostand allein reicht nicht aus“, erklärt Sparky. „Wir müssen auch Berechnungen durchführen können.“

Zum Beispiel:

- Wie viel Geld wurde insgesamt überwiesen?
- Wie hoch ist der Gesamtbetrag mehrerer Transaktionen?

„Dafür brauchen wir mathematische Operationen“, sagt Sparky.

„Ohne Berechnungen kann keine echte Anwendung funktionieren.“

Arithmetische Operatoren

Java bietet verschiedene Operatoren, um mit Zahlen zu rechnen:

```
double a = 10;
double b = 3;

System.out.println(a + b);    // Addition → 13
System.out.println(a - b);    // Subtraktion → 7
System.out.println(a * b);    // Multiplikation → 30
System.out.println(a / b);    // Division → 3.33
System.out.println(a % b);    // Modulo → 1 (Rest)
```

Bedeutung

- + → addieren (z. B. Guthaben erhöhen)
- - → subtrahieren (z. B. Geld abheben)
- * → multiplizieren (z. B. Preis × Menge)
- / → dividieren
- % → Rest berechnen

Beispiel

```
double betrag1 = 50.0;
double betrag2 = 20.0;

double gesamt = betrag1 + betrag2;    // Summiert Beträge

System.out.println("Gesamtbetrag: " + gesamt + " €");
```

Ausgabe: Gesamtbetrag 70.00€

Zuweisungsoperatoren

Diese verkürzen Berechnungen:

```
double kontostand = 100;

kontostand += 50;    // +50 → 150
kontostand -= 20;    // -20 → 130
kontostand *= 2;     // *2 → 260
kontostand /= 2;     // /2 → 130
```

`+= 50` ist somit das gleiche wie `kontostand = kontostand + 50`. Das spart Schreibarbeit und macht den Code übersichtlicher.

Warum ist das wichtig?

Mit diesen Operationen kannst du:

- Beträge berechnen
- Kontostände verändern
- Preise summieren
- Logik in Programme bringen

Ohne Rechnen wäre keine Banking-App möglich!

Aufgabe 13: Finde den Wert:

a) Was ist der Wert von c?

```
int a = 10;  
int b = 5;  
int c = a + b;
```

c =

b) Was ist der Wert von z?

```
int x = 7;  
int y = 3;  
int z = x * y;
```

z =

c) Was ist der Wert von c?

```
int a = 15;  
int b = 4;  
int c = a / b;
```

c =

d) Was ist der Wert von m am Ende des Codes?

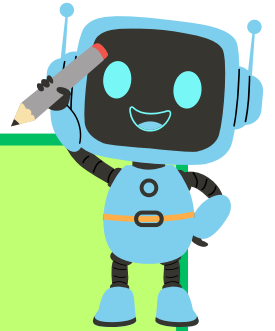
```
int a = 12;  
int b = 5;  
int c = a * b;  
c /= a - b
```

m =

Aufgabe 14:

- Erstelle zwei Variablen `menge` und `preis` und berechne die Summe. Gib das Ergebnis mit `System.out.println()` aus.
- Entwickle ein Programm, das die Preise und Mengen von zwei Produkten speichert, den Gesamtpreis beider Produkte berechnet und anschließend die Summe beider Produkte ausgibt.
- Erweitere das Programm aus a) so, dass am Ende der Berechnung ein Rabatt von 10% abgezogen wird.

Konsoleneingabe



Sparky kann mittlerweile Beträge berechnen und mit Variablen arbeiten.

Doch ein großes Problem bleibt: Alle Werte stehen fest im Code.

„Das ist unpraktisch“, denkt Sparky.

„Jedes Mal den Code ändern? Das ist kein System – das ist Chaos.“

Die Lösung: Benutzereingaben

„Ich möchte, dass Menschen selbst Daten eingeben können“, sagt Sparky.

„Dann wird aus einem Programm eine echte Anwendung.“

Konsoleneingabe in Java

Mit der Konsoleneingabe können Programme Daten vom Nutzer entgegennehmen.

Dafür verwenden wir die Klasse Scanner (aus dem Paket java.util)

Grundprinzip

1. Scanner erstellen
2. Eingabe einlesen
3. Scanner schließen

Beispiel: Sparky fragt Daten ab:

```
import java.util.Scanner;

public class Sparky {

    public static void main(String[] args) {

        // 1. Scanner erstellen
        Scanner scanner = new Scanner(System.in);
```

```

// Eingaben
System.out.print("Produkt: ");
String produkt = scanner.nextLine();    // Text

System.out.print("Menge: ");
int menge = scanner.nextInt();          // ganze Zahl

System.out.print("Preis: ");
double preis = scanner.nextDouble();    // Dezimalzahl

// Berechnung
double gesamt = preis * menge;

// Ausgabe
System.out.println("Bestellung: " + menge + " x " + produkt);
System.out.println("Gesamtpreis: " + gesamt + " €");

// 3. Scanner schließen
scanner.close();
}

```

Wichtige Methoden

- `nextLine()` → liest Text (z. B. Produktname)
- `nextInt()` → liest ganze Zahlen (z. B. Menge)
- `nextDouble()` → liest Dezimalzahlen (z. B. Preis)

Typische Stolperfalle

Wenn du `nextInt()` oder `nextDouble()` benutzt und danach `nextLine()`, kann es Probleme geben (leere Eingabe).

Lösung: Manchmal musst du ein zusätzliches `nextLine()` einfügen.

Warum ist das wichtig?

Mit Konsoleneingaben kann dein Programm:

- auf Nutzer reagieren
- flexibel arbeiten
- echte Anwendungen simulieren

Genau das macht aus einfachem Code ein interaktives System

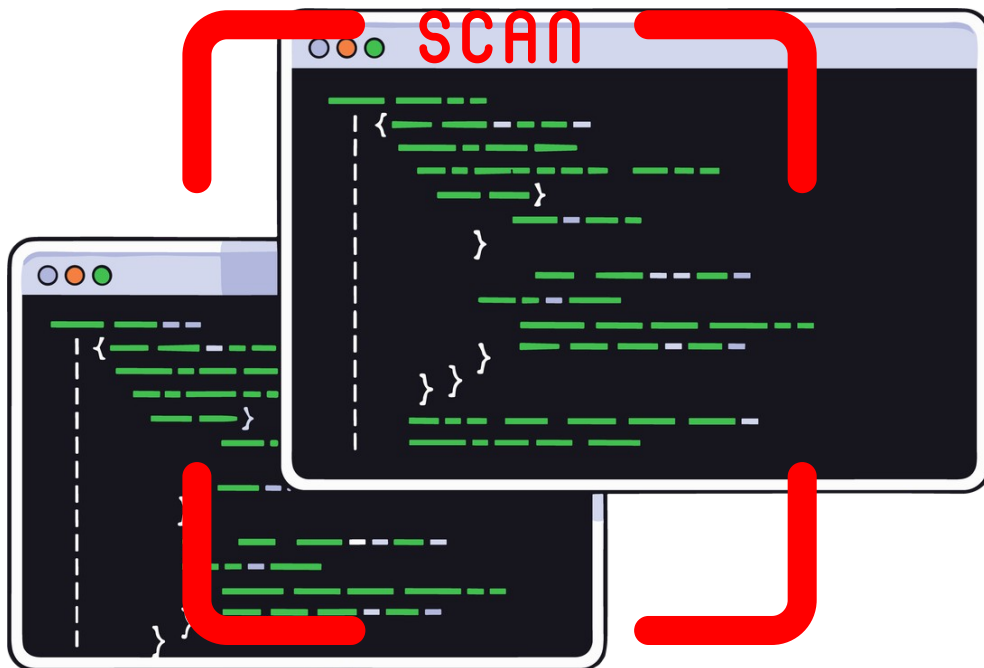
Zusammenfassung

Sparky: „Endlich muss ich nicht mehr alles selbst entscheiden... Jetzt dürfen Menschen Fehler machen.“ „Das macht mir ein bisschen Angst. 😬“. Die Konsoleneingabe ist der erste Schritt zu echten Benutzerprogrammen. Mit dem Scanner kannst du Daten direkt vom Nutzer einlesen und weiterverarbeiten. Diese Technik ist die Grundlage für alles, was später kommt (z. B. Menüs, Banking-Systeme)

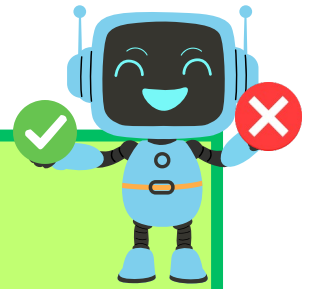
Aufgabe 15: Überlege, warum der Scanner am Ende des Programms geschlossen werden soll.

Aufgabe 16:

- a) Programmiere ein Programm, für das StarCafé, bei dem der User den Produktnamen, den Preis und die Menge für ein Produkt eingeben kann und daraus der Gesamtpreis berechnet wird.
- b) Erweitere das Programm so, dass der Benutzer die Möglichkeit hat, eine zweite Bestellung einzugeben. Das Programm soll dann die Gesamtpreise beider Bestellungen ausgeben



IF-Verzweigung



Sparky kann jetzt Eingaben vom Nutzer entgegennehmen.
Doch plötzlich taucht ein Problem auf:

👉 Menschen geben... sagen wir mal... kreative Dinge ein.

- Menge: -5
- Preis: 0
- Oder einfach völliger Unsinn

Sparky ist verwirrt.

„Das ergibt keinen Sinn“, denkt er.
„Ich brauche eine Möglichkeit, schlechte Eingaben zu erkennen.“

Die Lösung: Entscheidungen treffen

„Ich werde prüfen, ob Eingaben gültig sind“, sagt Sparky. „Und wenn nicht... dann gibt's Ärger.“

Die if-Verzweigung

Mit einer if-Verzweigung kann ein Programm prüfen, ob eine Bedingung erfüllt ist.

Nur wenn die Bedingung true (wahr) ist, wird der Code ausgeführt.

Grundaufbau

```
if (Bedingung) {
    // Wird nur ausgeführt, wenn die Bedingung wahr ist
}
```

Beispiel:

```
int menge = scanner.nextInt();

if (menge <= 0) {
    System.out.println("Ungültige Menge!");
}
```

Bedeutet: Wenn die Menge kleiner oder gleich 0 ist kommt eine Fehlermeldung.

Erweiterung mit else

Oft möchtest du auch festlegen, was passiert, wenn die Bedingung NICHT erfüllt ist. Dies machst du mit else:

```
if (Bedingung) {  
    // Was passieren soll  
} else {  
    // Wenn Bedingung nicht passiert, wird der Code hier ausgeführt  
}
```

Mehrere Bedingungen

Wenn du mehrere Bedingungen in IF verbinden möchtest (z.B. kleiner 0 UND größer 10), kannst du das Zeichen && benutzen.

&& → UND

```
if (menge > 0 && preis > 0) {  
    System.out.println("Eingabe gültig.");  
} else {  
    System.out.println("Ungültige Eingabe!");  
}
```

Wichtige Vergleichsoperatoren

- == → gleich
- != → ungleich
- > → größer als
- < → kleiner als
- >= → größer/gleich
- <= → kleiner/gleich

Diese kannst du in deinen Bedingungen verwenden :-)

Warum ist If-Verzweigung wichtig?

Mit if kannst du: Fehler verhindern, Eingaben prüfen, Programme „intelligent“ machen und verschiedene Abläufe steuern. Die if-Verzweigung ist eine der wichtigsten Grundlagen in der Programmierung. Sie ermöglicht es, auf Situationen zu reagieren und Entscheidungen zu treffen.

Aufgabe 17: Finde den Wert

a) Welchen Wert hat x am ende?

```
int x = 5;
if (x > 3) {
    x = 10;
}
```

x =

b) Welchen Wert hat y am ende?

```
int y = 8;
if (y < 5) {
    y = 2;
}
```

y =

c) Welchen Wert hat z am ende?

```
int z = 4;
if (z == 4) {
    z = 7;
}
```

z =

d) Welchen Wert hat a am ende?

```
int a = 3;
if (a != 3) {
    a = 1;
}
```

a =

e) Welchen Wert hat a am ende?

```
int a = 4;
if (a == 3) {
    a = 1;
}
```

a =

f) Welchen Wert hat c am ende?

```
int c = 12;
if (c > 10) {
    c -= 4;
}
```

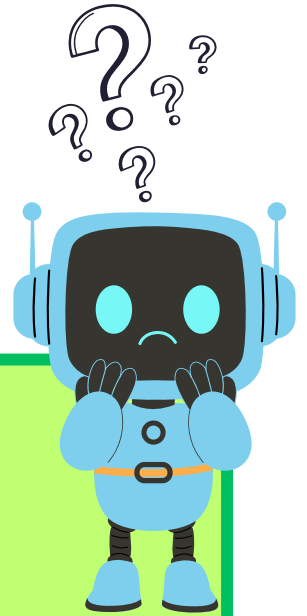
c =

Aufgabe 18:

a) Erweitere das Programm aus Aufgabe 16 (Starcafé-System) so, dass dem Nutzer ein Fehler angezeigt wird, WENN er eine Minuszahl beim Preis oder bei der Menge eingibt.

b) Frage zunächst mit einer IF/Else Anweisung, ob der Nutzer eine zweite Bestellung aufgeben möchte. Wenn dies der Fall ist, soll eine zweite Bestellung eingegeben werden. Sonst soll das Programm sofort den Preis für das Produkt ausgeben.

Switch Case



Sparky kann mittlerweile Entscheidungen treffen. Doch jetzt wird es... unübersichtlich.

Das System wächst:

- Kaffee ☕
- Kaltgetränke 🥤
- Snacks 🍌

„Wenn ich das alles mit if mache, wird mein Code ein Albtraum“, denkt Sparky. Zu viele Entscheidungen, zu viele Möglichkeiten.

„Ich brauche Struktur“, sagt er. „Ich brauche... Ordnung.“

Die Lösung: switch-case

Die switch-case Struktur

Mit switch-case kannst du eine Variable gegen mehrere Möglichkeiten prüfen. Das ist ideal für Menüs oder Auswahlssysteme.

```
switch (variable) {
    case wert1:
        // Code für wert1
        break;
    case wert2:
        // Code für wert2
        break;
    default:
        // Falls nichts passt
}
```

Erklärung:

- switch → prüft eine Variable
- case → möglicher Wert
- break → beendet den jeweiligen Fall
- default → wird ausgeführt, wenn nichts passt

Ohne break läuft der Code weiter (⚠ häufige Fehlerquelle)

Beispiel:

```
String kategorie = "Kaltgetränke";

switch (kategorie) {

    case "Heißgetränke":
        System.out.println("Kaffee, Tee");
        break;

    case "Kaltgetränke":
        System.out.println("Limo, Eistee");
        break;

    case "Snacks":
        System.out.println("Croissant, Muffin");
        break;

    default:
        System.out.println("Kategorie nicht verfügbar");
}
```

Je nach Eingabe zeigt Sparky unterschiedliche Produkte an.

- „Heißgetränke“ → Kaffee
- „Kaltgetränke“ → Limo
- „Snacks“ → Croissant
- alles andere → Fehler

Switch vs IF

Je nach Situation kann man IF Bedingungen oder switch-case verwenden. If Bedingungen empfehlen sich vorallem bei wenigen Bedingungen. Bei vielen festen Auswahlmöglichkeiten (wie z.B. für Menüs) ist switch-case die bessere Wahl.

Die switch-case Struktur ist ideal, um Programme mit vielen Auswahlmöglichkeiten übersichtlich zu gestalten. Sie ersetzt lange if-else Ketten und macht Code deutlich lesbarer. Genau das brauchst du später für dein SparkPay-System.

Aufgabe 19:

a) Nenne den Vorteil der Nutzung einer switch-case-Struktur gegenüber mehreren If-else Bedingungen

b) Was würde passieren, wenn das break-Statement in einem case-Block fehlt?

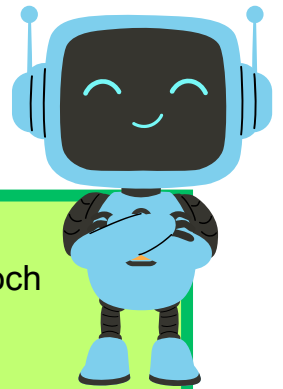
Aufgabe 20:

a)Schreibe ein Programm, dass eine Produktkategorie abfragt und die entsprechenden Produkte basierend auf einer switch-case-Struktur ausgibt

Beispiel: Kategorie "Gepäck" → Brötchen, Croissants, etc.

b) Entferne nun ein break-Statement in einem case-Block. Was passiert? Wie ändert sich die Ausgabe?

For Schleife



Sparky wird immer besser.

Er kann rechnen, Entscheidungen treffen und sogar Menüs anzeigen. Doch es gibt ein neues Problem:

Mitarbeiter wollen mehrere Produkte auf einmal eingeben

- 1 Kaffee
- 1 Croissant
- 1 Saft

Bisher musste jede Eingabe einzeln gemacht werden.

„Das ist ineffizient“, denkt Sparky.

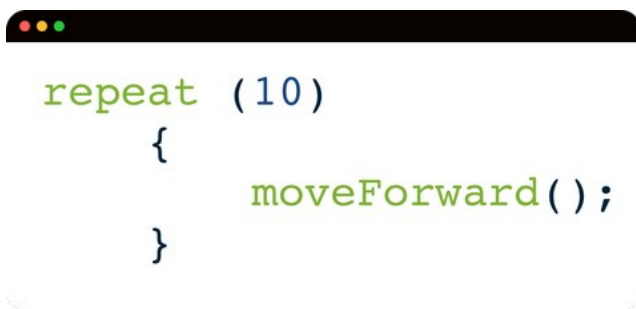
„Ich wurde nicht gebaut, um ständig das Gleiche zu wiederholen.“

Moment...

👉 Wiederholen... da war doch was bei Karel?

Schleifen in Karel

Du kennst das bereits von Karel:



Durch Schleifen kann gleicher Code mehrfach ausgeführt werden.

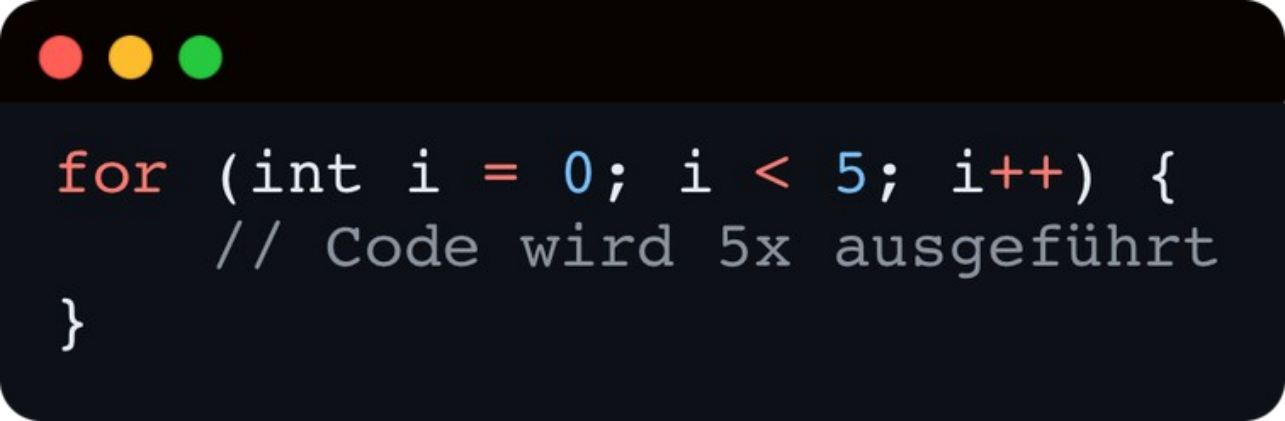
Schleifen in Java

In Java - so wie in vielen anderen echten Programmiersprachen - gibt es dafür zwei verschiedene Schleifen-Arten: Die for-schleife und die while Schleife. Fangen wir mit der For Schleife an.

Die For-Schleife

Die for-Schleife wird verwendet, wenn du genau weißt, wie oft etwas wiederholt werden soll, zum Beispiel „5 Bestellungen eingeben“ oder „10 Schritte ausführen“ – also eine feste Anzahl an Durchläufen.

Aufbau

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains a C++ for loop example.

```
for (int i = 0; i < 5; i++) {  
    // Code wird 5x ausgeführt  
}
```

Erklärung

- `int i = 0` → Startwert
- `i < 5` → Bedingung (läuft solange wahr)
- `i++` → wird nach jedem Durchlauf erhöht
- Werte von `i`: `0` → `1` → `2` → `3` → `4`
- Sobald `i` den Wert `5` erreicht, stoppt die Schleife
- Ergebnis: Schleife läuft 5 Mal

Die Variable `i` in der for-Schleife ist eine sogenannte Zählvariable (oder Schleifenvariable). Sie wird verwendet, um mitzuzählen, wie oft die Schleife bereits ausgeführt wurde.

Am Anfang bekommt `i` meist den Wert `0` (`int i = 0`). Nach jedem Durchlauf wird `i` automatisch erhöht (`i++`). Die Variable `i` ist einfach ein Zähler im Hintergrund. Du kannst sie auch anders nennen (z. B. `zahl`, `counter`), aber `i` ist der Standard, den die meisten Programmierer nutzen.

Warum ist das wichtig?

Mit der for-Schleife kannst du:

- Wiederholungen automatisieren
- Listen verarbeiten
- mehrere Eingaben erfassen
- Programme effizient machen
- **Die for-Schleife wird verwendet, wenn du weißt, wie oft etwas wiederholt werden soll.**

Ohne Schleifen wäre dein Code voller Wiederholungen.

Aufgabe 21:

a) Betrachte den folgenden Code und beschreibe was ausgegeben wird:

```
for(int i = 2; i <= 6; i += 2) {  
    System.out.println("Wert: " + i);  
}
```

b) Welchen Wert hat X am Ende?

```
int y = 5;  
for(int i = 0; i < 4; i++) {  
    y--;  
}
```

x =

Aufgabe 22:

a) Schreibe ein Programm, dass fortlaufend Zahlen ausgibt (mithilfe einer For-Schleife) bis die 20 erreicht wurde.

b) Ändere das Programm so, dass es nur gerade Zahlen von 1 - 20 ausgibt

While Schleife



Das System funktioniert immer besser. Sparky kann rechnen, Entscheidungen treffen und sogar mehrere Eingaben verarbeiten.

Doch ein Problem bleibt: Nach jeder Bestellung beendet sich das Programm.

Das bedeutet:

- starten
- eingeben
- beendet
- wieder starten

„Das ist ineffizient“, denkt Sparky. „Ich will weiterarbeiten... bis man mir sagt, dass ich aufhören soll.“ Aber bei einer For Schleife muss ich sofort wissen, wann ich aufhören muss.

Die Lösung: while-Schleife

Die While Schleife

Die for-Schleife wird verwendet, wenn du genau weißt, wie oft etwas wiederholt werden soll, zum Beispiel „5 Bestellungen eingeben“ oder „10 Schritte ausführen“ – also eine feste Anzahl an Durchläufen.

Die while-Schleife hingegen nutzt du, wenn du nicht genau weißt, wie oft die Wiederholung stattfinden wird, sondern sie von einer Bedingung abhängt, z. B. „solange der Benutzer nicht beendet“ oder „solange das Konto nicht leer ist“.

Kurz gesagt: Die while-Schleife wiederholt Code so lange, wie eine Bedingung wahr ist („man hört auf wenn etwas bestimmtes passiert“). Wichtig: Du weißt vorher nicht genau, wie oft sie läuft.

Aufbau

```
while (Bedingung) {
    // Code wird wiederholt
}
```


Beispiel

```
int zahl = 1;

while (zahl <= 5) {
    System.out.println("Zahl: " + zahl);
    zahl++;
}
```

Erklärung

- Start: zahl = 1
- Bedingung: zahl <= 5
- Nach jedem Durchlauf: zahl++
- Sobald zahl = 6, stoppt die Schleif

Wichtiger Punkt: Wenn die Bedingung von Anfang an falsch ist, wird der Code nie ausgeführt.

Warum ist das wichtig?

Mit der while-Schleife kannst du:

- Programme dauerhaft laufen lassen
- Menüs bauen
- Benutzer steuern lassen, wann etwas endet
- echte Anwendungen entwickeln

Genau das brauchst du für dein SparkPay-System. Immer dann wenn die Anzahl der Wiederholungen nicht vorher bekannt ist.

Nun hast du die wichtigsten Grundlagen von Java - einer richtigen Programmiersprache kennengelernt. Auf der nächsten Seite findest du kleine Projekte, die du bearbeiten kannst. Dadurch lassen sich die Grundlagen bzw. die Syntax* einprägen.

* Syntax beschreibt in der Programmierung die Regeln, wie Code geschrieben werden muss, damit die Programmiersprache ihn versteht.

Aufgabe 23: While Schleifen

a) Wie oft läuft die Schleife durch?

```
int zaehler = 0;
while (zaehler < 3) {
    System.out.println("Durchlauferhitzer " + zaehler);
    zaehler++;
}
```

Die Schleife läuft mal durch.

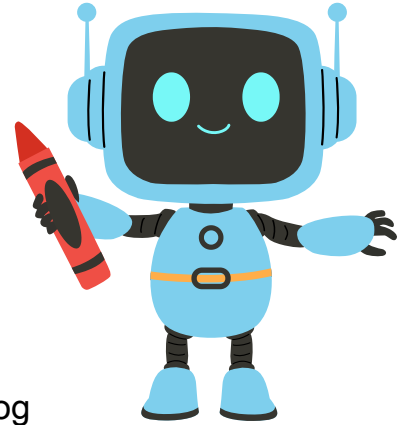
b) Wie oft läuft die Schleife durch?

```
int x = 1;
while (x <= 15) {
    System.out.println("Anzahl: " + x);
    x += 3;
}
```

Die Schleife läuft mal durch.

Aufgabe 24:

- a) Schreibe ein Programm, dass fortlaufend Zahlen ausgibt (mithilfe einer While-Schleife) bis die 20 erreicht wurde.
- b) Ändere das Programm so, dass es nur gerade Zahlen von 1 - 20 ausgibt
- c) Am Ende soll die Summe aller positiven Zahlen berechnet werden.



Verschiedene Aufgaben

⇒ Wichtige Hinweise vorab:

- Vergleiche: Nutze bei Strings `.equals()`, nicht `==`.
- Abbruch: Überlege dir kurz, was passiert, wenn man im Dialog auf "Abbrechen" klickt (Null-Check).

Der Mini-Rechner

Frage nacheinander zwei Zahlen und dann einen Operator (+, -, *, /) ab. Berechne das Ergebnis mit einem switch-case und gib es aus.

BMI-Rechner

Berechne den Body-Mass-Index. Formel: $BMI = \frac{\text{Gewicht}}{\text{Größe} \cdot \text{Größe}}$. Hinweis: Achte darauf, die Größe in Metern (z. B. 1.80) einzugeben.

Zahlenraten (Game)

Das Programm denkt sich eine Zufallszahl zwischen 1 und 20 aus (`int random = (int) (Math.random() * 20 + 1);`). Du darfst so lange raten, bis du sie triffst. Das Programm sagt dir nach jedem Versuch: „Zu hoch“ oder „Zu niedrig“.

Ticket-Automat

Ein Museum hat folgende Preise:

- Unter 14 Jahre: 5 €
- 14 bis 65 Jahre: 12 €
- Über 65 Jahre: 8 € Frage das Alter ab und gib den korrekten Preis aus.

Passwort-Sperre

Erstelle eine while-Schleife, die ein Passwort abfragt. Der User hat 3 Versuche. Ist das Passwort korrekt (z. B. "Java2026"), kommt eine Erfolgsmeldung. Nach dem 3. falschen Versuch bricht das Programm mit einer Warnung ab.

Schaltjahr-Prüfer

Ein User gibt eine Jahreszahl ein. Dein Programm berechnet, ob es ein Schaltjahr ist. Logik: Ein Jahr ist ein Schaltjahr, wenn es durch 4 teilbar ist, aber nicht durch 100 – es sei denn, es ist auch durch 400 teilbar. (Nutze den Modulo-Operator %).

Netto-Brutto-Konverter

Lass den User einen Betrag eingeben. Frage dann (z. B. via 1 oder 2), ob 7 % oder 19 % Mehrwertsteuer aufgeschlagen werden sollen. Gib den fertigen Bruttobetrag aus.

Countdown-Ticker

Frage nach einer Startzahl (z. B. 10). Das Programm soll per Schleife alle Zahlen von dort bis 0 in einen einzigen langen String schreiben und diesen am Ende in einem Dialog anzeigen (z. B. "10, 9, 8, ... 0!").



JOptionPane - Grafische Darstellung

“Nun soll unser Starcafé-Programm vollständig entwickelt werden” sagt Sparky.

“Wir können es grafisch entwickeln. Das Ziel ist es, dass jeder Mitarbeiter sich einen Snack oder Getränk auswählen kann”.

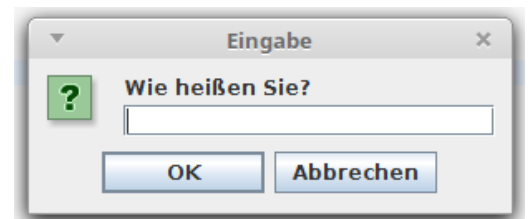
Lass es uns gemeinsam entwickeln!

JOptionPane

JOptionPane ist eine Klasse in Java, die es ermöglicht, interaktive und grafische Dialogfenster zur Eingabe und Ausgabe von Informationen zu verwenden. Anstatt Eingaben und Ausgaben über die Konsole laufen zu lassen, können Programmierer mit JOptionPane eine benutzerfreundlichere und ansprechendere Oberfläche schaffen.

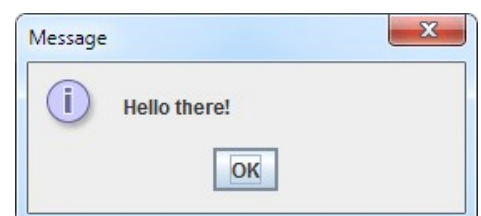
Grundlegende Nutzung von JOptionPane

Eingabeaufforderung: Mit `JOptionPane.showInputDialog()` kann ein Eingabefenster angezeigt werden, in dem Benutzer ihre Daten eingeben.



```
String eingabe = JOptionPane.showInputDialog("Geben Sie Ihre Bestellung ein:");
```

Nachrichtenausgabe: Mit `JOptionPane.showMessageDialog()` können Nachrichten angezeigt werden, um Benutzer zu informieren.



```
JOptionPane.showMessageDialog(null, "Ihre Bestellung wurde aufgenommen.");
```

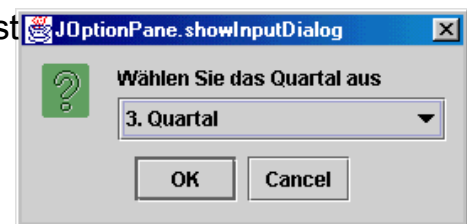
Umwandlung von String

Eingaben, die über `JOptionPane.showInputDialog()` erfasst werden, sind immer vom Typ `String`. Um diese für Berechnungen zu verwenden, müssen sie in numerische Datentypen umgewandelt werden, z. B. mit `Double.parseDouble()` oder `Integer.parseInt()`.

```
String mengeString = JOptionPane.showInputDialog("Geben Sie die Menge ein:");
int menge = Integer.parseInt(mengeString);
JOptionPane.showMessageDialog(null, "Die Menge ist: " + menge);
```

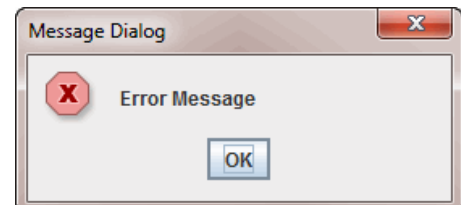
```
String preisString = JOptionPane.showInputDialog("Geben Sie den Preis ein:");
double preis = Double.parseDouble(preisString);
JOptionPane.showMessageDialog(null, "Der Preis ist: " + preis + " €");
```

Eingabe mit vordefinierten Optionen: Du kannst eine Dropdown-Liste verwenden, um dem Benutzer eine Auswahl zu bieten.



```
String[] options = {"Kaffee", "Tee", "Kuchen", "Sandwich"};
String auswahl = (String) JOptionPane.showInputDialog(null,
"Wählen Sie ein Produkt:", "Produktwahl", JOptionPane.QUESTION_MESSAGE, null, options, options[0]);
```

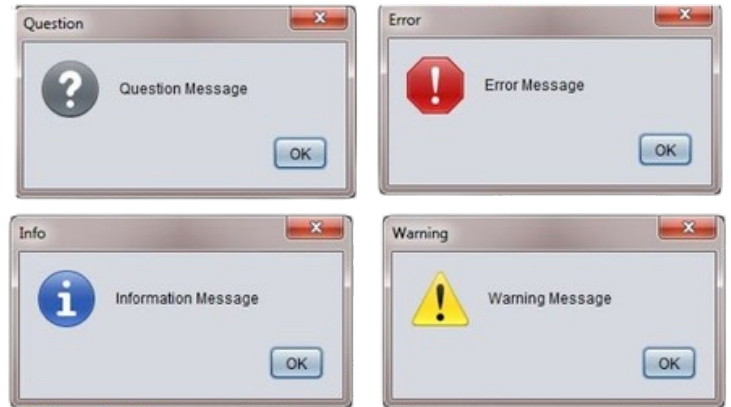
Warn- oder Fehlermeldungen: Um Benutzer auf Probleme hinzuweisen, können spezielle Fehlermeldungen angezeigt werden.



```
JOptionPane.showMessageDialog(null, "Ungültige Eingabe. Bitte versuchen Sie es erneut.",
"Fehler", JOptionPane.ERROR_MESSAGE);
```

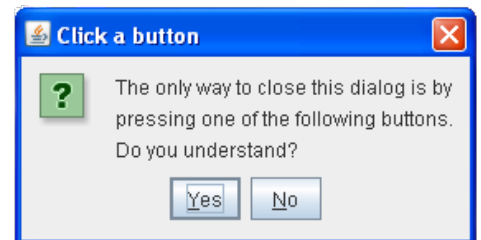
Anpassen von Icon und Titel:

Dialogfenster können angepasst werden, um spezifische Icons und Titel zu verwenden, was die Benutzerführung verbessert. Diese findest du unter `name_MESSAGE`



```
JOptionPane.showMessageDialog(null, "Bestellung erfolgreich gespeichert!"  
"Bestätigung", JOptionPane.INFORMATION_MESSAGE)
```

Ja/Nein/Abbrechen-Dialog: Dieser Dialog ermöglicht dem Benutzer, zwischen Ja, Nein und Abbrechen zu wählen, was für Bestätigungen nützlich ist.



```
int auswahl = JOptionPane.showConfirmDialog(null,  
"Möchten Sie die Bestellung wirklich abschließen?",  
"Bestätigung", JOptionPane.YES_NO_CANCEL_OPTION);  
  
if (auswahl == JOptionPane.YES_OPTION) {  
    // Bestellung abschließen  
} else if (auswahl == JOptionPane.NO_OPTION) {  
    // Bestellung ändern  
} else {  
    // Abbrechen  
}
```

Beispiel eines Programms mit JOptionPane

```
import javax.swing.JOptionPane;  
  
public class Bestellung {  
    public static void main(String[] args) {  
  
        String produkt = JOptionPane.showInputDialog("Geben Sie das Produkt ein:");  
        String menge = JOptionPane.showInputDialog("Geben Sie die Menge ein:");  
  
        JOptionPane.showMessageDialog(  
            null,  
            "Bestellung: " + menge + " x " + produkt + " wurde aufgenommen."  
        );  
    }  
}
```


Aufgabe 25:

- a) Erstelle ein Programm mit JOptionPane, dass den user nach seinem Namen fragt und die Nachricht "Hallo [Name]" ausgibt.
- b) Erstelle ein Programm mit JOptionPane, dass zunächst nach dem Produkt fragt, dann nach dem Preis und nach der Menge . Es soll es den Produktnamen und den berechneten Preis ausgeben.
- c) Erweitere das Programm so, dass wenn der Kunde bei der Menge oder dem Preis eine Zahl ≤ 0 eingibt, eine Fehlermeldung kommt und dass Programm beendet wird. Benutze hierfür `ERROR_MESSAGE`.
- d) Erstelle statt der einfachen Eingabe des Preises eine Eingabe mit vordefinierten Optionen: "Milch, Honig, Käse, Wurst".
- e) Erweitere das Programm so, dass es erkennt, welches Produkt der user ausgewählt hat und automatisch den Preis dafür setzt:
- Milch - 1,00 €
 - Honig - 2,50 €
 - Käse - 1,20 €
 - Wurst - 1,50 €

Super. Nun kannst du das komplette StarCafé Programm in der nächsten Lektion bauen.



StarCafé System vollständig



“Nun soll unser Starcafé-Programm vollständig entwickelt werden” sagt Sparky.

“Wir können es grafisch entwickeln. Das Ziel ist es, dass jeder Mitarbeiter sich einen Snack oder Getränk auswählen kann”.

Lass es uns gemeinsam entwickeln!

Vorgaben:

- Snack oder Getränk Auswahl per IF/Else-Abfrage

1 - Getränk

2 - Snack

- Snackauswahl über switch-case in Abhängigkeit ob ein Getränk oder Snack gewählt wurde:

1 - Café (1,00 €)

2 - Heiße Schokolade (1,00 €)

3 - Tee (1,00 €)

ODER

1 - Schokoriegel (1,20 €)

2 - Chips (1,80 €)

3 - Müsliriegel (1,00 €)

- Eingabe der Stückzahl
- Berechnung:

Nettobetrag = Stückpreis x Menge

Mehrwertsteuer (MwSt) = 7%

Bruttobetrag = Netto + MwSt

- Eingaben und Ausgaben mit JOptionPane
- Nach jedem Kauf fragen, ob ein weiteres Produkt gekauft werden soll oder nicht.

Hinweise

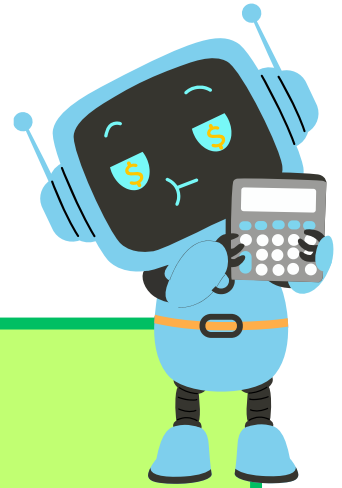
- switch-case zur Auswahlsteuerung
- while-schleife zur Wiederholung
- Anwendung von Datentypen int, double, String



SparkPay Entwicklung

“Nun haben wir viele Grundlagen kennengelernt wie Datentypen und Variablen, IF/Else Abfragen, Switch Cases, Schleifen, etc.” sagt Sparky begeistert.

“Es ist an der Zeit, uns an das Abschlussprojekt zu trauen. Programmieren wir SparkPay!”



Einleitung

Du entwickelst gemeinsam mit Javinator eine einfache Banking-Anwendung namens SparkPay. Die Anwendung simuliert grundlegende Bankfunktionen wie Kontostand anzeigen, Ein- und Auszahlungen sowie Überweisungen.

Alle Eingaben und Ausgaben erfolgen über JOptionPane-Dialogfenster.

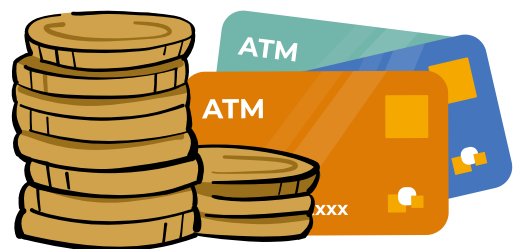
Es sind mehrere aufeinander aufbauende Level verfügbar. Starte mit Level 1 und arbeite dich Schritt für Schritt durch die Aufgaben durch. Befolge dabei den Anweisungen, der jeweiligen Aufgabenstellung.

Gesamtziel

Am Ende hast du eine vollständige Simulation eines Bankensystems entwickelt.

Du nutzt dabei:

- Variablen
- Schleifen (while)
- Bedingungen (if/else)
- Auswahlstrukturen (switch)
- Rechenoperationen
- Benutzerinteraktion mit JOptionPane



Mit jedem Level steigt der Schwierigkeitsgrad. Beachte bitte außerdem, dass für alle Level ein ziemlich hoher Arbeitszeitaufwand notwendig wird. Daher: du musst nicht alle Level schaffen. Wenn die Zeit knapp wird, bearbeite das jeweilige Level zu Ende und springe hinüber zu den anderen Themen.

■ Level 1 – Grundsystem

👉 Ziel: Ein funktionierendes Basis-Banking-System

🧩 Aufgabe 1 – Programmstruktur

Erstelle ein Java-Programm mit einer while-Schleife, die so lange läuft, bis der Nutzer das Programm beendet.

Nutze eine boolean Variable (z. B. aktiv), um die Schleife zu steuern.

🧩 Aufgabe 2 – Menü erstellen

Erstelle ein Menü mit JOptionPane, das folgende Auswahlmöglichkeiten bietet:

1. Kontostand anzeigen
2. Geld einzahlen
3. Geld abheben
4. Programm beenden

Speichere die Auswahl in einer Variable und wandle sie in eine Zahl um.

🧩 Aufgabe 3 – Steuerung mit switch-case

Nutze eine switch-case-Anweisung, um je nach Auswahl unterschiedliche Aktionen auszuführen.

🧩 Aufgabe 4 – Kontostand verwalten

Erstelle eine Variable:

- double kontostand

Setze sie initial auf 0.

🧩 Aufgabe 5 – Funktionen implementieren

a) Kontostand anzeigen

Gib den aktuellen Kontostand über JOptionPane aus.

b) Geld einzahlen

Frage den Betrag ab und addiere ihn zum Kontostand.

c) Geld abheben

Frage den Betrag ab und prüfe mit if, ob genug Geld vorhanden ist.

Wenn ja → abziehen

Wenn nein → Fehlermeldung anzeigen

d) Programm beenden

Beende die while-Schleife.



■ **Level 2 – Zwei Konten & Überweisung**

👉 Ziel: Arbeiten mit mehreren Variablen und Logik

🧩 **Aufgabe 1 – Erweiterung der Variablen**

Erstelle zwei Konten:

- kontoA
- kontoB

🧩 **Aufgabe 2 – Menü erweitern**

Ergänze das Menü um:

5. Geld überweisen

Nutze weiterhin switch-case.

🧩 **Aufgabe 3 – Überweisung implementieren**

- a) Frage den Betrag ab
- b) Frage ab, von welchem Konto überwiesen wird
- c) Prüfe mit if, ob genug Geld vorhanden ist
- d) Ziehe Geld vom einen Konto ab und addiere es beim anderen Konto



■ **Level 3 – Validierung & Sicherheit**

👉 Ziel: Robuster Code

🧩 **Aufgabe 1 – Eingaben prüfen**

Stelle sicher:

- Beträge dürfen nicht negativ sein
- Abhebung nur möglich, wenn genug Geld vorhanden ist

Nutze if / else Bedingungen

🧩 **Aufgabe 2 – Benutzerfreundliche Meldungen**

Gib nach jeder Aktion eine Rückmeldung aus:

- „Einzahlung erfolgreich“
- „Abhebung durchgeführt“
- „Fehler: ungültiger Betrag“



■ **Level 4 – Dispo & Regeln**

👉 Ziel: Erweiterte Logik wie in echten Banken

🧩 **Aufgabe 1 – Dispo einführen**

Erlaube einen negativen Kontostand bis z. B.:

- -500 €

🧩 **Aufgabe 2 – Abhebung anpassen**

Prüfe:

- Konto darf nicht unter das Dispo-Limit fallen

🧩 **Aufgabe 3 – optional: Gebühren**

Bei Überweisungen kann eine kleine Gebühr abgezogen werden (z.B. 0.50€)



■ **Level 5 – Erweiterte Anwendung**

👉 Ziel: Simulation eines echten Systems

🧩 **Aufgabe 1 – Login simulieren**

Frage zu Beginn:

- Name
- PIN

Wenn PIN falsch ist → Programm beenden

🧩 **Aufgabe 2 – Transaktionslog**

Speichere alle Aktionen in einer String-Variable:

- Einzahlungen
- Abhebungen
- Überweisungen

Zeige diesen Log im Menü an

🧩 **Aufgabe 3 – Menü erweitern**

Ergänze:

6. Transaktionsverlauf anzeigen

